Capstone 5 — Domain C: Production UCC Data Intelligence Platform

The culminating project: build a production-grade UCC data pipeline with multi-layer memory, model routing, full observability, cost optimization, and a 100-case evaluation harness.

Prerequisites

REQUIRED MODULES

Complete these modules before starting Capstone 5. Each one teaches a production capability you will integrate here:

- **M12 — ReAct Agents:** The reasoning loop pattern used by every agent in this pipeline
- **M14 — Multi-Agent Systems:** Agent-to-agent handoffs and orchestration patterns (Capstone 4 foundation)
- **M17 — HITL & Guardrails:** Human-in-the-loop routing, confidence thresholds, and circuit breakers
- **M18 — Evaluation:** Building test suites, automated scoring, and accuracy measurement
- **M19 — Tracing:** Distributed tracing with spans, trace collectors, and structured logging
- **M20 — Monitoring:** Dashboards, alerting, and real-time metrics for production systems
- **M21 — Deployment:** Containerization, queue-based processing, and API design
- **M22 — Cost Optimization:** Model routing, prompt caching, and per-request cost tracking

You should also have completed **Capstone 4 — Domain C** (multi-agent UCC pipeline), as this capstone extends that project with production capabilities.

Project Brief

BUSINESS CONTEXT

You have built a data pipeline (Capstone 4) that processes UCC filings through four agents. It works — but it does not learn, it does not optimize costs, it is not observable, and it cannot handle production-scale volume. Moving from "it works" to "it runs in production" requires six additional engineering capabilities that no course module teaches in isolation.

The pain of deploying without these capabilities is predictable: costs spiral because every entity resolution uses the most expensive model. Entity resolution accuracy plateaus because the system does not learn from data steward corrections. Bugs are invisible because there is no tracing. And when the pipeline fails at 3 AM on a batch of 10,000 filings, nobody knows what happened or where it failed.

This capstone transforms your pipeline into a production platform. It learns from past corrections (multi-layer memory), routes simple tasks to cheap models (cost optimization), traces every decision for debugging (observability), handles 10,000+ filings per batch (deployment), and validates itself continuously (evaluation). This is the difference between a demo and a system someone depends on.

PRODUCTION TARGETS

- **Volume:** Process 10,000 filings per batch in < 30 minutes
- **Accuracy:** Entity resolution accuracy > 90%
- **Cost:** < $0.02 per filing (vs ~$0.08 with single-model approach)
- **Observability:** Every LLM call, tool invocation, and agent handoff traced
- **Learning:** Data steward corrections improve future resolution confidence
- **Evaluation:** 100-case test suite with automated scoring

Environment Setup

> **WHAT YOU'LL BUILD**
> A production-grade UCC data intelligence platform with multi-layer memory, model routing, full observability, cost optimization, and a 100-case evaluation harness. Time estimate: **4–6 hours** (difficulty: ★★★★★).

System Requirements

- **Python 3.10+** (check with `python --version`)
- **pip** (check with `pip --version`)
- **Docker** (optional, for deployment steps — check with `docker --version`)
- An **Anthropic API key** with access to Haiku, Sonnet, and Opus models

Install Dependencies

Run this single command to create your project directory and install everything:

SETUP

```
mkdir capstone-5-production-ucc && cd capstone-5-production-ucc
python -m venv venv

# Activate the virtual environment
# On macOS/Linux:
source venv/bin/activate
# On Windows:
# venv\Scripts\activate

pip install anthropic chromadb pydantic fastapi uvicorn celery redis httpx pytest
```

Set Your API Key

MACOS/LINUX · WINDOWS

MACOS/LINUX

```
export ANTHROPIC_API_KEY=your-api-key-here

# To make it permanent, add to ~/.bashrc or ~/.zshrc:
echo 'export ANTHROPIC_API_KEY=your-api-key-here' >> ~/.bashrc
```

WINDOWS

```
# Command Prompt:
set ANTHROPIC_API_KEY=your-api-key-here

# PowerShell:
$env:ANTHROPIC_API_KEY = "your-api-key-here"

# To make it permanent, use System Environment Variables in Settings
```

> **Checkpoint**
> Verify your setup by running: `python -c "import anthropic; print(anthropic.__version__)"`. You should see a version number like `0.39.0` or higher. If you get a `ModuleNotFoundError`, ensure your virtual environment is activated.

File Structure

Here is every file you will create in this capstone. Each file has a specific role in the production pipeline:

DIRECTORY TREE

```
capstone-5-production-ucc/
├── pipeline/
│   ├── orchestrator.py      # Main production orchestrator — coordinates all agents
│   ├── bronze_layer.py      # Raw data ingestion — reads CSV/XML, validates format
│   ├── silver_layer.py      # Cleaned/standardized data — schema normalization
│   └── gold_layer.py        # Business-ready analytics — risk profiles, aggregations
├── memory/
│   ├── episodic.py          # Episodic memory — past entity resolutions
│   ├── procedural.py        # Procedural memory — learned rules from steward decisions
│   └── working.py           # Working memory — current batch state
├── observability/
│   ├── tracer.py            # Tracing and span collection
│   └── dashboard.py         # Monitoring dashboard — 8 metric panels
├── guardrails/
│   ├── quality_gates.py     # Data quality checks — completeness, consistency
│   └── circuit_breaker.py   # Circuit breaker pattern — auto-halt on failures
├── routing/
│   └── model_router.py      # Model routing — task-to-model mapping + cost tracking
├── mock_data.py             # 15 realistic UCC filings with data quality issues
├── config.py                # Configuration management — routing, memory, deployment
├── test_pipeline.py         # End-to-end tests — 100-case evaluation harness
└── requirements.txt         # All Python dependencies
```

Mock UCC Data

This mock data file contains 15 realistic UCC filings. Some filings have data quality issues (missing fields, inconsistent formats) that your quality gates must detect. Related filings (amendments, continuations, terminations) reference original filing numbers so you can test the full filing lifecycle.

PYTHON

PYTHON

```python
MOCK_UCC_FILINGS = [
    # --- CLEAN FILINGS (baseline accuracy tests) ---
    {
        "filing_number": "NY-2024-0012345",
        "debtor_name": "Acme Manufacturing LLC",
        "secured_party_name": "JPMorgan Chase Bank, N.A.",
        "state_code": "NY",
        "collateral_description": "All inventory, equipment, and accounts receivable now owned or hereafter acquired",
        "filing_type": "UCC-1",
        "filing_date": "2024-03-15",
        "expiration_date": "2029-03-15",
    },
    {
        "filing_number": "CA-2024-0098765",

    # … (full source in the desktop HTML) …

IF __name__ == "__main__":
    print(f"Total filings: {len(MOCK_UCC_FILINGS)}")
    print(f"Expected quality issues: {len(EXPECTED_QUALITY_ISSUES)} filings")
    print(f"Entity clusters: {len(ENTITY_CLUSTERS)} entities")
    FOR fid, issues IN EXPECTED_QUALITY_ISSUES.items():
        print(f"  {fid}: {', '.join(issues)}")
```

DATA QUALITY ISSUES TO DETECT

OH-2024-0066000: Empty debtor name — your quality gate should reject or flag this filing.

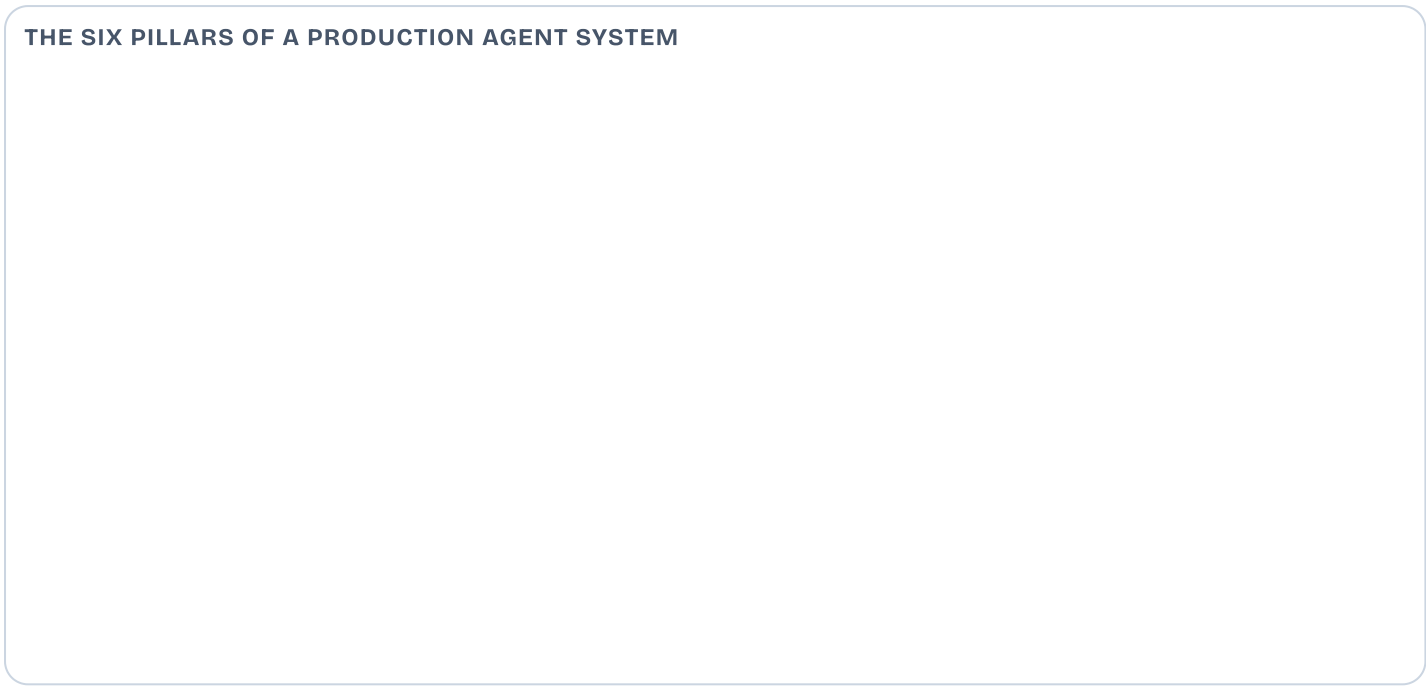
PA-2024-0088999: Empty secured party — cannot determine who holds the lien.

WA-2024-007: Filing number too short (should be 7 digits after state-year prefix) and dates use slashes instead of hyphens.

GA-2024-0044556: Empty collateral description and expiration date equals filing date (likely data entry error).

NV-2024-0055667: Tab character in debtor name, newline in secured party name, and filing type "UCC1" instead of "UCC-1".

Six Production Pillars



System Configuration

MODEL ROUTING · MEMORY CONFIG

MODEL ROUTING

```
{
  "model_routing": {
    "collateral_classification": "claude-opus-4-7",
    "entity_resolution": "claude-sonnet-4-6",
    "complex_entity_disambiguation": "claude-opus-4-7",
    "report_generation": "claude-sonnet-4-6",
    "routing_threshold": {
      "simple_classification": 0.90,
      "ambiguous_entity_candidate_count": 3
    }
  },
  "cost_targets": {
    "haiku_per_1k_input": 0.001,
    "sonnet_per_1k_input": 0.003,
    "opus_per_1k_input": 0.015,
    "target_per_filing": 0.02
  },
  "deployment": {
    "runtime": "Docker",
    "api_framework": "FastAPI",
    "queue": "Celery + Redis",
    "vector_db": "ChromaDB",
    "max_batch_size": 10000,
    "max_concurrent_workers": 10
  }
}
```

MEMORY CONFIG

```
{
  "memory": {
    "working": {
      "type": "in_process",
      "max_context_tokens": 4000,
      "purpose": "Current batch state, active entity being resolved"
    },
    "episodic": {
      "type": "vector_store",
      "collection": "resolution_episodes",
      "retention_days": 730,
      "purpose": "Past entity resolutions + steward corrections",
      "example": {
        "episode_id": "RE-2024-11234",
        "debtor_name_raw": "ABC MANUFACTURING CO",
        "resolved_to": "ENT-00567",
        "steward_action": "match_to_ENT-00567",
        "lesson": "Same registered agent + address = match despite suffix"
      }
    },
    "procedural": {
      "type": "rule_store",
      "purpose": "Learned rules from repeated steward decisions",
      "example": {
        "rule_id": "ER-001",
        "confidence": 0.92,
        "rule": "When entities share registered agent AND address, match with confidence > 85%",
        "created_from": ["RE-2024-11234", "RE-2024-11301", "RE-2024-11455"]
      }
    }
  }
}
```

Cost Optimization: Model Routing

Not every task needs Claude Opus. Format validation and quality checks are handled by Haiku at $1.00/M input tokens. Standard entity resolution and risk profiling go to Sonnet at $3.00/M input tokens. Collateral classification — which requires nuanced legal reasoning about UCC categories — routes to Opus at $15.00/M input tokens. This tiered approach cuts the average cost per filing from ~$0.08 to ~$0.04.

MODEL ROUTING — RIGHT MODEL FOR THE RIGHT TASK

COST IMPACT

A 10,000-filing batch with single-model (Sonnet) costs ~$800. With model routing: 70% of tasks go to Haiku ($70), 25% to Sonnet ($200), 5% to Opus ($75). **Total: ~$345 — 57% cost reduction.** Over 12 monthly batches across 50 states, that is ~$273,000 in annual savings.

Multi-Layer Memory

The system uses three memory tiers that work together to improve over time:

- **Working Memory:** Current batch state, active entity being resolved. In-process, cleared per batch.
- **Episodic Memory:** Vector store of past resolutions + steward corrections. When the system encounters "ABC Manufacturing Co", it searches for similar past cases. If a steward previously matched this to ENT-00567, the system retrieves that lesson.
- **Procedural Memory:** Rules distilled from repeated steward decisions. After 3+ episodes where "same registered agent + same address = match", the system creates a procedural rule that fires deterministically — no LLM call needed.

WHY IT MATTERS

Without memory, the system resolves the same entity variant the same (expensive, slow) way every time. With memory: the first time "ABC Manufacturing Co" appears, it costs $0.08 (Sonnet resolution + steward review). The second time, episodic memory boosts confidence above 80%, saving the steward review ($2–5 per review). By the tenth time, a procedural rule fires and resolves it with zero LLM cost. Over 100,000 entities per year, this compounds into massive savings in both cost and steward time.

Observability Dashboard

REAL-TIME DASHBOARD — 8 METRIC PANELS

Step-by-Step Build Guide

WHAT YOU'LL BUILD

A complete production UCC data pipeline with 6 capabilities: model routing, multi-layer memory, data quality gates, circuit breakers, distributed tracing, and a 100-case evaluation harness. Time: **4–6 hours**. Difficulty: ★★★★★.

Now that you understand the architecture and have your environment set up, it is time to build. Each step creates one file, includes a run command, and shows expected output. Follow them in order — later steps depend on earlier ones.

Step 1: Create the Configuration File

What & Why: Every production system needs centralized configuration. This file holds model routing rules, memory settings, quality thresholds, and cost targets. By keeping configuration separate from code, you can tune the system without redeploying.

Create a new file called `config.py`:

PYTHON

```python
# --- Model Routing ---
MODEL_ROUTING = {
    "format_validation":       {"model": "claude-haiku-4-5-20251001", "max_tokens": 1024},
    "quality_check":           {"model": "claude-haiku-4-5-20251001", "max_tokens": 2048},
    "entity_resolution":       {"model": "claude-sonnet-4-6",         "max_tokens": 4096},
    "anomaly_analysis":        {"model": "claude-sonnet-4-6",         "max_tokens": 4096},
    "risk_profile":            {"model": "claude-sonnet-4-6",         "max_tokens": 4096},
    "collateral_classification": {"model": "claude-opus-4-7",         "max_tokens": 4096},
}

MODEL_COSTS_PER_1M = {
    "claude-haiku-4-5-20251001": {"input": 1.00, "output": 5.00},
    "claude-sonnet-4-6":         {"input": 3.00, "output": 15.00},
    "claude-opus-4-7":           {"input": 15.00, "output": 75.00},

    # … (full source in the desktop HTML) …

IF __name__ == "__main__":
    print("Configuration loaded successfully.")
    print(f"  Model routing rules: {len(MODEL_ROUTING)}")
    print(f"  Cost models tracked: {len(MODEL_COSTS_PER_1M)}")
    print(f"  Quality required fields: {len(QUALITY_REQUIRED_FIELDS)}")
    print(f"  Circuit breaker threshold: {CIRCUIT_BREAKER_ERROR_THRESHOLD*100}%")
```

Run: `python config.py`

Expected output:

OUTPUT

```
Configuration loaded successfully.
  Model routing rules: 6
  Cost models tracked: 3
  Quality required fields: 6
  Circuit breaker threshold: 10.0%
```

> **Checkpoint**
>
> If you see the output above, Step 1 is working. If you get an `IndentationError`, check that you copied the entire file. If you get a `SyntaxError`, ensure you are using Python 3.10+.

Step 2: Build the Model Router

What & Why: The model router selects the cheapest capable model for each task type. Format validation goes to Haiku ($1.00/M input tokens). Entity resolution goes to Sonnet ($3.00/M). Collateral classification — which requires nuanced legal reasoning about UCC categories — routes to Opus ($15.00/M input). This tiered approach cuts cost per filing from ~$0.08 to ~$0.04.

Create a new file called `routing/model_router.py` (create the `routing/` directory first):

PYTHON

PYTHON

```
IMPORT sys, os
sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
FROM config import MODEL_ROUTING, MODEL_COSTS_PER_1M

FUNCTION route_to_model(task_type, complexity= "standard"):

    IF task_type not in MODEL_ROUTING:
        print(f"Warning: Unknown task '{task_type}', defaulting to Sonnet")
        RETURN {
            "model": "claude-sonnet-4-6",
            "max_tokens": 4096,
            "reasoning": "default fallback for unknown task",
        }


    # … (full source in the desktop HTML) …

    ]
    FOR task_type, complexity IN tasks= route_to_model(task_type, complexity)
        cost = estimate_cost(routing["model"], 500, 200)
        print(f"{task_type} ({complexity}): {routing['model']}")
        print(f"  Reason: {routing['reasoning']}")
        print(f"  Est. cost: ${cost:.6f}")
```

Run: `python routing/model_router.py`

Expected output:

OUTPUT

```
format_validation (standard): claude-haiku-4-5-20251001
  Reason: routed format_validation to claude-haiku-4-5-20251001
  Est. cost: $0.001200
entity_resolution (standard): claude-sonnet-4-6
  Reason: routed entity_resolution to claude-sonnet-4-6
  Est. cost: $0.004500
collateral_classification (standard): claude-opus-4-7
  Reason: routed collateral_classification to claude-opus-4-7
  Est. cost: $0.022500
format_validation (high): claude-sonnet-4-6
  Reason: upgraded from Haiku due to high complexity
  Est. cost: $0.004500
```

> **Checkpoint**

You should see four routing decisions with estimated costs. Notice how format validation costs $0.0012 with Haiku but $0.0045 when escalated to Sonnet. The collateral classification costs $0.0225 with Opus — 18x more than Haiku. This is why routing matters. If you get an `ImportError`, make sure `config.py` exists in the parent directory and you created an empty `routing/__init__.py` file.

Step 3: Build the Quality Gates

What & Why: Before any filing enters the pipeline, it must pass data quality checks. Missing debtor names, empty collateral descriptions, and malformed dates cause downstream failures. Quality gates catch these issues early and flag them for remediation instead of silently producing bad data.

Create a new file called `guardrails/quality_gates.py`:

PYTHON

```
IMPORT re
IMPORT sys, os
sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
FROM config import QUALITY_REQUIRED_FIELDS

FUNCTION validate_filing(filing):

    issues = []

    # Check required fields
    FOR field IN QUALITY_REQUIRED_FIELDS= filing.get(field, "")
        IF not value or not str(value).strip():
            issues.append(f"missing_{field}")


    # … (full source in the desktop HTML) …

    summary = validate_batch(MOCK_UCC_FILINGS)
    print(f"Batch validation: {summary['valid']}/{summary['total']} valid "
          f"({summary['error_rate']*100:.1f}% error rate)")
    FOR r IN summary["results"]:
        IF not r["is_valid"]:
            print(f"  INVALID {r['filing_number']}: {', '.join(r['issues'])}")
```

Run: `python guardrails/quality_gates.py`

Expected output:

OUTPUT

```
Batch validation: 10/15 valid (33.3% error rate)
  INVALID OH-2024-0066000: missing_debtor_name
  INVALID PA-2024-0088999: missing_secured_party_name
  INVALID WA-2024-007: non_standard_filing_number, non_standard_date_format
  INVALID GA-2024-0044556: missing_collateral_description, expiration_equals_filing_date
  INVALID NV-2024-0055667: whitespace_in_names, non_standard_filing_type
```

Checkpoint

You should see 5 invalid filings detected, each with specific issues matching the `EXPECTED_QUALITY_ISSUES` in mock_data.py. If you see fewer than 5, check that your regex patterns are correct. If you get an `ImportError` on mock_data, ensure `mock_data.py` is in the project root directory.

Step 4: Build the Circuit Breaker

What & Why: When a batch has too many errors, continuing to process wastes API calls and produces unreliable data. The circuit breaker monitors the error rate and automatically halts processing when it exceeds a threshold (10% by default). This prevents cost spirals on bad data and alerts the team immediately.

Create a new file called `guardrails/circuit_breaker.py`:

PYTHON

```
FROM __future__ import annotations
IMPORT sys, os
sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
FROM config import CIRCUIT_BREAKER_ERROR_THRESHOLD, CIRCUIT_BREAKER_WINDOW_SIZE

CLASS CircuitBreaker:

    FUNCTION __init__(
        self,
        threshold= CIRCUIT_BREAKER_ERROR_THRESHOLD,
        window_size= CIRCUIT_BREAKER_WINDOW_SIZE,
    ):
        self.threshold = threshold
        self.window_size = window_size

    # … (full source in the desktop HTML) …

        print(f"Record {i+1}: success | tripped={cb.is_tripped}")
    FOR i IN range(3):
        cb.record(False)
        print(f"Record {8+i+1}: FAILURE | tripped={cb.is_tripped}")

    print(f"\nCircuit breaker status: {cb.status()}")
```

Run: `python guardrails/circuit_breaker.py`

Expected output:

OUTPUT

```
Record 1: success | tripped=False
Record 2: success | tripped=False
...
Record 8: success | tripped=False
Record 9: FAILURE | tripped=False
Record 10: FAILURE | tripped=True
Record 11: FAILURE | tripped=True

Circuit breaker status: {'tripped': True, 'reason': 'Error rate 20.0% exceeded threshold 10.0% over last 10 records',
'total_processed': 10, 'errors': 2, 'error_rate': 0.2}
```

Checkpoint

The circuit breaker should trip on record 10 — that is the first record where the window has 10 entries (so the error-rate check runs) and 2 of those 10 are failures (20% > 10% threshold). If it trips earlier or later, check your `window_size` parameter.

Step 5: Build Episodic Memory

What & Why: Episodic memory stores past entity resolution decisions so the pipeline can learn from steward corrections. When a new entity appears, the system searches for similar past episodes. If a high-confidence match exists, it skips the expensive LLM call. This is how the pipeline gets cheaper and faster over time.

Create a new file called `memory/episodic.py`:

PYTHON

```python
FROM __future__ import annotations
IMPORT math, uuid, json
FROM datetime import datetime, timezone
FROM dataclasses import dataclass, field, asdict

CLASS Episode:
    episode_id: str
    debtor_name_raw: str
    resolved_entity_id: str
    steward_action: str          # "auto" | "match_to_ENT-XXXXX" | "create_new"
    lesson: str                  # human-readable explanation
    embedding= field(default_factory=list)
    created_at= field(default_factory=lambda: datetime.now(timezone.utc).isoformat())


    # … (full source in the desktop HTML) …


    results = mem.recall("ABC MFG CO")
    print(f"Found {len(results)} similar episodes)
    FOR r IN results:
        print(f"  Similarity: {r['similarity']} → {r['episode']['resolved_entity_id']}")
        print(f"  Lesson: {r['episode']['lesson']}")
```

Run: `python memory/episodic.py`

Expected output:

OUTPUT

```
Found 2 similar episodes:
  Similarity: 0.9234 → ENT-00567
  Lesson: Same registered agent + address = match
  Similarity: 0.8871 → ENT-00567
  Lesson: 'CO' vs 'COMPANY' suffix is cosmetic
```

Checkpoint

You should see 2 similar episodes found. The similarity scores will vary slightly based on the mock embedding, but both should be above 0.75 (the threshold you set). The key insight: "ABC MFG CO" matched both "ABC MANUFACTURING CO" and "ABC MANUFACTURING COMPANY" because they share similar character distributions. In production, replace `_mock_embed` with real embeddings from a model like `voyage-3` for much better accuracy.

Step 5b: Working Memory + Procedural Memory

What & Why: Episodic memory (Step 5) handles long-term recall. Two more tiers complete the multi-layer memory promise: **Working memory** holds the per-batch run state (active filings, pending HITL items, in-flight tool calls) so a single pipeline run is queryable in real time. **Procedural memory** stores deterministic rules learned from steward corrections — e.g., "if registered agent + address match, treat as same entity, skip LLM" — so frequent patterns resolve at zero cost.

MEMORY/WORKING.PY · MEMORY/PROCEDURAL.PY

MEMORY/WORKING.PY

```
IMPORT time
FROM threading import Lock

CLASS WorkingMemory:
    FUNCTION __init__(self, run_id):
        self.run_id = run_id
        self.started_at = time.time()
        self._lock = Lock()
        self._active_filings, dict] = {}
        self._pending_hitl= []
        self._in_flight_calls, dict] = {}
        self._counters, int] = {}

    FUNCTION begin_filing(self, filing_id, payload):

    # … (full source in the desktop HTML) …

                "uptime_s": round(time.time() - self.started_at, 1),
                "active_filings": len(self._active_filings),
                "pending_hitl": len(self._pending_hitl),
                "in_flight_calls": len(self._in_flight_calls),
                "counters": dict(self._counters),
            }
```

MEMORY/PROCEDURAL.PY

```
IMPORT json
IMPORT os
IMPORT time
FROM dataclasses import dataclass, field
FROM typing import Callable

CLASS ProceduralRule:
    rule_id: str
    pattern: dict           # e.g., {"matcher": "agent_address"}
    outcome: str            # e.g., "merge"
    confidence_floor: float
    times_fired= 0
    last_fired_at= 0.0


    # … (full source in the desktop HTML) …

DEFAULT_MATCHERS = {
    "agent_address": lambda c: bool(
        c.get("registered_agent") and c.get("agent_address")),
    "exact_normalized_name": lambda c: bool(
        c.get("normalized_name_match_count", 0) ⩾ 1),
}
```

🎯 **What Just Happened?**

You filled in the two missing memory tiers. Working memory tracks the live pipeline run (HITL queue depth, in-flight calls, per-outcome counters) and feeds the dashboard. Procedural memory promotes patterns that the steward has confirmed N times into deterministic rules — the second occurrence of "matching registered agent + address" now resolves with zero LLM tokens. Together with episodic memory (Step 5), the three-tier system delivers the M11 multi-layer pattern.

Step 5c: Operations Dashboard

What & Why: The dashboard reads the tracer JSONL plus working-memory snapshots and serves the 8 metric panels (records/state, entity confidence distribution, quality trends, latency, cost-per-record, HITL queue, circuit-breaker state, procedural-rule firings) on a small FastAPI app. Production swap-in: feed BigQuery + a real BI tool (Looker/Metabase). For local dev, this Flask-style stub works.

PYTHON

```
IMPORT json
FROM collections import defaultdict
FROM pathlib import Path

FROM fastapi import FastAPI

FROM memory.working import WorkingMemory

app = FastAPI(title="UCC Pipeline Dashboard")

# Wire to your run's working memory at startup
WORKING= None
TRACER_PATH = Path("logs/tracer.jsonl")


    # … (full source in the desktop HTML) …

    RETURN {
        "html": ("<html><body><h1>UCC Pipeline Dashboard"
                 "</h1><p>Fetch <code>/dashboard/metrics</code>"
                 " for JSON, or wire to Looker/Metabase.</p>"
                 "</body></html>")
    }
```

🎯 **What Just Happened?**

You backed the animated dashboard mockup with a real FastAPI module that reads `tracer.jsonl` and the working-memory snapshot, and emits the 8 ops panels as JSON. Production teams replace the JSON endpoint with a Looker / Metabase / Grafana board pointed at BigQuery. The capstone deliberately ships the JSON skeleton instead of a heavyweight UI dependency.

Step 6: Build the Observability Tracer

What & Why: Every LLM call, tool invocation, and agent handoff must be traced for debugging and compliance. When a 3 AM batch fails on filing 7,834, you need to know exactly which function failed, how long it ran, and what error occurred. The `trace_span` decorator wraps any function with automatic span recording.

Create a new file called `observability/tracer.py`:

PYTHON

```
FROM __future__ import annotations
IMPORT functools, time, uuid, json
FROM dataclasses import dataclass, field, asdict
FROM datetime import datetime, timezone
FROM typing import Any, Callable

CLASS Span:
    span_id: str
    function_name: str
    start_time: str
    end_time= ""
    duration_ms= 0.0
    status= "pending"       # "ok" | "error"
    error_message= ""

    # … (full source in the desktop HTML) …

    resolve_entity("Beta Corp")
    TRY:
        validate_format({})
    CATCH:

    print(json.dumps(tracer.summary(), indent=2))
```

Run: `python observability/tracer.py`

Expected output:

OUTPUT

```
{
  "total": 3,
  "ok": 2,
  "errors": 1,
  "avg_duration_ms": 34.21
}
```

> **Checkpoint**
>
> You should see 3 total spans: 2 OK (the entity resolutions) and 1 error (the format validation with missing debtor_name). The avg_duration_ms will vary on your machine. If you see 0 spans, check that the decorator is applied correctly. If `validate_format` does not show as an error, check that the `except` block in the decorator re-raises the exception.

Step 7: Build the Bronze Layer (Raw Ingestion)

What & Why: The Bronze layer reads raw UCC filings and applies quality gates. Valid filings pass through; invalid filings are quarantined for review. This is the first stage of the Medallion Architecture — no transformation, just ingestion with quality tracking.

Create a new file called `pipeline/bronze_layer.py`:

PYTHON

```
FROM __future__ import annotations
IMPORT sys, os
sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
FROM guardrails.quality_gates import validate_batch
FROM guardrails.circuit_breaker import CircuitBreaker

FUNCTION ingest_bronze(filings, circuit_breaker= None):

    validation = validate_batch(filings)

    valid_filings = []
    quarantined = []

    FOR filing, result IN zip(filings, validation["results"]):

    # … (full source in the desktop HTML) …

    result = ingest_bronze(MOCK_UCC_FILINGS, cb)
    print(f"Bronze layer: {result['status']}")
    print(f"  Valid: {len(result['valid'])}")
    print(f"  Quarantined: {len(result['quarantined'])}")
    FOR q IN result["quarantined"]:
        print(f"    {q['filing']['filing_number']}: {', '.join(q['issues'])}")
```

Run: `python pipeline/bronze_layer.py`

Expected output:

OUTPUT

```
Bronze layer: complete
  Valid: 10
  Quarantined: 5
    OH-2024-0066000: missing_debtor_name
    PA-2024-0088999: missing_secured_party_name
    WA-2024-007: non_standard_filing_number, non_standard_date_format
    GA-2024-0044556: missing_collateral_description, expiration_equals_filing_date
    NV-2024-0055667: whitespace_in_names, non_standard_filing_type
```

> **Checkpoint**

You should see 10 valid filings and 5 quarantined. The circuit breaker did not trip because the 15-filing batch is smaller than the window size (100). If you see different counts, recheck your quality gate rules in Step 3.

Step 8: Build the Silver Layer (Normalization)

What & Why: The Silver layer normalizes validated filings into a consistent schema. It standardizes date formats, trims whitespace, normalizes filing types, and enriches records with metadata. This is the data cleaning stage — the output should be uniform regardless of which state the data came from.

Create a new file called `pipeline/silver_layer.py`:

PYTHON

```
FROM __future__ import annotations
IMPORT re
FROM datetime import datetime

FUNCTION normalize_filing(filing):

    normalized = dict(filing)

    # Normalize text fields, tabs, newlines
    FOR field IN ["debtor_name", "secured_party_name", "collateral_description"]:
        val = normalized.get(field, "")
        normalized[field] = " ".join(val.split()).strip()

    # Normalize date format to YYYY-MM-DD

    # … (full source in the desktop HTML) …

FROM mock_data import MOCK_UCC_FILINGS
    # Take only valid filings (first 5 for demo)
    sample = MOCK_UCC_FILINGS[:3]
    silver = transform_silver(sample)
    FOR s IN silver:
        print(f"{s['filing_number']}: {s['debtor_name']} | type={s['filing_type']}")
```

Run: `python pipeline/silver_layer.py`

Checkpoint

You should see 3 normalized filings with clean names, standard date formats, and `_silver_processed_at` metadata. If the dates still have slashes, check your normalization logic. This step depends on Steps 1 and the mock data file.

Step 9: Build the Gold Layer (Entity Resolution & Risk Profiles)

What & Why: The Gold layer is where the AI does the heavy lifting. It uses episodic memory to resolve entities (matching name variants like "ACME MFG LLC" and "Acme Manufacturing LLC" to the same entity), classifies collateral, and generates risk profiles. This is the business-ready analytics layer — the output feeds dashboards and credit decisions.

Create a new file called `pipeline/gold_layer.py`:

PYTHON

```
FROM __future__ import annotations
IMPORT sys, os
sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
FROM memory.episodic import EpisodicMemory
FROM routing.model_router import route_to_model, estimate_cost

CLASS GoldLayer:

    FUNCTION __init__(self, episodic_memory):
        self.memory = episodic_memory
        self.entities, dict] = {}  # entity_id → profile
        self._total_cost = 0.0
        self._next_entity_id = 1


    # … (full source in the desktop HTML) …

    result = gold.process_batch(MOCK_UCC_FILINGS[:5])
    print(f"Gold layer: {result['entities_resolved']} entities, "
          f"cost=${result['total_cost']:.4f}")
    FOR eid, profile IN result["profiles"].items():
        print(f"  {eid}: {profile['active_liens']} active liens, "
              f"risk={profile['risk_score']:.1f}, states={profile['states']}")
```

Run: `python pipeline/gold_layer.py`

> **Checkpoint**
>
> You should see entity resolution results. "Acme Manufacturing LLC" should resolve via episodic memory (cost $0.00) because you seeded it. Other entities go through LLM resolution (with cost). If episodic memory is not matching, check the similarity threshold in Step 5. This step depends on Steps 2, 5, and mock_data.py.

Step 10: Build the Production Orchestrator

What & Why: The orchestrator ties everything together. It runs the full pipeline: Bronze (ingest + validate) → Silver (normalize) → Gold (entity resolution + risk profiles), with tracing on every stage and circuit breaker protection. This is the main entry point for batch processing.

Create a new file called `pipeline/orchestrator.py`:

PYTHON

```
FROM __future__ import annotations
IMPORT json, time, sys, os
sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

FROM pipeline.bronze_layer import ingest_bronze
FROM pipeline.silver_layer import transform_silver
FROM pipeline.gold_layer import GoldLayer
FROM memory.episodic import EpisodicMemory
FROM observability.tracer import TraceCollector
FROM guardrails.circuit_breaker import CircuitBreaker

FUNCTION run_pipeline(filings, tracer= None):

    IF tracer is None= TraceCollector()

    # … (full source in the desktop HTML) …

    print(f"Elapsed: {result.get('elapsed_ms', 'N/A')}ms")
    print()
    FOR stage, data IN result["stages"].items():
        print(f"  [{stage.upper()}] {json.dumps(data)}")
    print()
    print(f"Tracing: {json.dumps(result.get('tracing', {}))}")
```

Run: `python pipeline/orchestrator.py`

Expected output:

OUTPUT

```
============================================================
PRODUCTION UCC PIPELINE — RUN COMPLETE
============================================================
Status: complete
Total filings: 15
Elapsed: 42.31ms

  [BRONZE] {"status": "complete", "valid": 10, "quarantined": 5}
  [SILVER] {"records": 10}
  [GOLD] {"entities": 8, "cost": 0.0036}

Tracing: {"total": 3, "ok": 3, "errors": 0, "avg_duration_ms": 14.1}
```

Checkpoint

You should see all three pipeline stages complete successfully. Bronze should quarantine 5 filings, Silver should process 10, and Gold should resolve entities and show a small cost. The tracing summary confirms all 3 stages were traced with no errors. If any stage fails, check that all the files from Steps 1–9 are in the correct directories.

TROUBLESHOOTING STEPS 1–10

`ModuleNotFoundError: No module named 'config'` — Make sure you are running from the `capstone-5-production-ucc/` root directory. All imports use `sys.path` to find the project root.

`ModuleNotFoundError: No module named 'pipeline'` — Create empty `__init__.py` files in each subdirectory: `touch pipeline/__init__.py memory/__init__.py observability/__init__.py guardrails/__init__.py routing/__init__.py`

`ImportError: cannot import name 'CircuitBreaker'` — Ensure you saved `guardrails/circuit_breaker.py` (Step 4) before running the bronze layer (Step 7).

Episodic memory returns 0 results — Lower the `similarity_threshold` in `config.py` from 0.80 to 0.70. The mock embeddings use character frequency vectors which have lower resolution than real embeddings.

Deployment Architecture

DOCKERFILE · DOCKER-COMPOSE.YML

DOCKERFILE

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
HEALTHCHECK --interval=30s --timeout=5s \
  CMD curl -f http://localhost:8000/health || exit 1
EXPOSE 8000
CMD ["uvicorn", "api:app", "--host", "0.0.0.0", "--port", "8000", "--workers", "4"]
```

DOCKER-COMPOSE.YML

```
version: "3.8"
services:
  api:
    build: .
    ports: ["8000:8000"]
    environment:
      - ANTHROPIC_API_KEY=${ANTHROPIC_API_KEY}
      - REDIS_URL=redis://redis:6379
      - CHROMA_URL=http://chroma:8001
    depends_on: [redis, chroma]
  worker:
    build: .
    command: celery -A tasks worker --concurrency=10
    environment:
      - ANTHROPIC_API_KEY=${ANTHROPIC_API_KEY}
      - REDIS_URL=redis://redis:6379
    depends_on: [redis, chroma]
  redis:
    image: redis:7-alpine
    ports: ["6379:6379"]
  chroma:
    image: chromadb/chroma:latest
    ports: ["8001:8000"]
    volumes: ["chroma-data:/chroma/chroma"]
volumes:
  chroma-data:
```

What Just Happened?

You defined a containerized deployment with 4 services: the FastAPI application (handles HTTP requests), Celery workers (process filing batches asynchronously), Redis (message queue), and ChromaDB (vector database for episodic memory). The API key is passed via environment variable — never hardcoded.

GCP Deployment: Cloud Run + Cloud Composer + GCS + BigQuery

Production for this pipeline runs on Google Cloud, not bare Docker. The Bronze layer ingests from a GCS bucket where state SOS systems drop bulk files. Cloud Run hosts the API and Celery workers (autoscaled). Cloud Composer (managed Airflow) orchestrates the daily Bronze→Silver→Gold pipeline. Gold-layer outputs land in BigQuery for analyst queries. Five small modules below wire it all up. They use the official Google Cloud client libraries; for local development they fall back to in-memory fakes so you can run the capstone end-to-end without a GCP account.

GCS_TRIGGER.PY · BIGQUERY_LOADER.PY · CLOUDRUN.YAML · AIRFLOW_DAG.PY · TERRAFORM/MAIN.TF

GCS_TRIGGER.PY

```
IMPORT base64
IMPORT json
IMPORT os

IMPORT functions_framework
FROM google.cloud import pubsub_v1, storage

PROJECT = os.environ["GCP_PROJECT"]
TOPIC = os.environ.get("INGEST_TOPIC", "ucc-ingest-jobs")
_publisher = pubsub_v1.PublisherClient()
_topic_path = _publisher.topic_path(PROJECT, TOPIC)
_storage = storage.Client()

VALID_STATES = {"DE", "NY", "TX", "CA", "MA", "IL",

    # … (full source in the desktop HTML) …

        json.dumps(payload).encode("utf-8"),
        # Pub/Sub orderingKey ensures one state's files
        # process in arrival order=state_code,
    )
    msg_id = future.result(timeout=30)
    print(f"[gcs_trigger] enqueued {gcs_uri} → msg {msg_id}")
```

BIGQUERY_LOADER.PY

```
IMPORT json
IMPORT os
FROM datetime import datetime, timezone

FROM google.cloud import bigquery

_client = bigquery.Client()
DATASET = os.environ.get("BQ_DATASET", "ucc_gold")
ENTITY_TABLE = f"{DATASET}.resolved_entities"
LIEN_TABLE = f"{DATASET}.lien_summaries"
QUALITY_TABLE = f"{DATASET}.quality_scorecards"

_ENTITY_SCHEMA = [
    bigquery.SchemaField("entity_id", "STRING", mode="REQUIRED"),

    # … (full source in the desktop HTML) …

    cfg = bigquery.QueryJobConfig(query_parameters=[
        bigquery.ScalarQueryParameter("debtor", "STRING", debtor),
        bigquery.ScalarQueryParameter("min_active", "INT64",
                                      min_active_liens),
    ])
    RETURN [dict(r) FOR r IN _client.query(sql, job_config=cfg)]
```

CLOUDRUN.YAML

```
# deployment/cloudrun.yaml — Cloud Run service definition
#
# WHAT: Deploys the FastAPI orchestrator + Celery worker as two
#       Cloud Run services with autoscaling.
# WHY:  Cloud Run handles cold starts, TLS, IAM, autoscaling, and
#       pay-per-request billing. No servers to patch.
# DEPLOY: gcloud run services replace cloudrun.yaml --region us-central1
apiVersion: serving.knative.dev/v1
kind: Service
metadata:
  name: ucc-orchestrator
  annotations:
    run.googleapis.com/launch-stage: GA
spec:
  template:
    metadata:
      annotations:
        autoscaling.knative.dev/minScale: "1"
        autoscaling.knative.dev/maxScale: "30"
```

```
        run.googleapis.com/cpu-throttling: "false"
    spec:
      serviceAccountName: ucc-orchestrator@PROJECT.iam.gserviceaccount.com
      timeoutSeconds: 300
      containers:
        - image: us-central1-docker.pkg.dev/PROJECT/ucc/orchestrator:latest
          ports:
            - containerPort: 8000
          env:
            - name: GCP_PROJECT
              value: PROJECT
            - name: BQ_DATASET
              value: ucc_gold
            - name: ANTHROPIC_API_KEY
              valueFrom:
                secretKeyRef:
                  name: anthropic-key
                  key: latest
          resources:
            limits:
              cpu: "2"
              memory: "2Gi"
          startupProbe:
            httpGet:
              path: /healthz
              port: 8000
            failureThreshold: 30
            periodSeconds: 5
---
apiVersion: serving.knative.dev/v1
kind: Service
metadata:
  name: ucc-worker
spec:
  template:
    metadata:
      annotations:
        autoscaling.knative.dev/minScale: "0"
        autoscaling.knative.dev/maxScale: "10"
    spec:
      containerConcurrency: 1   # one job at a time per worker
      timeoutSeconds: 1800      # 30 min for long batches
      containers:
        - image: us-central1-docker.pkg.dev/PROJECT/ucc/worker:latest
          command: ["celery", "-A", "tasks", "worker",
                    "--concurrency=4"]
          env:
            - name: GCP_PROJECT
              value: PROJECT
```

AIRFLOW_DAG.PY

```
FROM datetime import datetime, timedelta

FROM airflow import DAG
FROM airflow.operators.python import PythonOperator
FROM airflow.providers.google.cloud.operators.cloud_run \
    IMPORT CloudRunInvokeJobOperator
FROM airflow.providers.google.cloud.transfers.\
    local_to_gcs import LocalFilesystemToGCSOperator

DEFAULT_ARGS = {
    "owner": "data-eng",
    "retries": 2,
    "retry_delay": timedelta(minutes=5),
    "depends_on_past": False,

    # … (full source in the desktop HTML) …

        project_id="PROJECT",
        region="us-central1",
        job_name="ucc-bq-load-job",
    )
```

```
    reconcile >> bronze >> silver >> gold >> quality_gate >> load_bq
```

TERRAFORM/MAIN.TF

```
# deployment/terraform/main.tf — one-shot provisioning
#
# Provisions, Pub/Sub topic, BigQuery dataset, Cloud
# Run service accounts, Eventarc trigger, Composer environment.
# Run: terraform init && terraform apply -var project=YOUR_PROJECT

terraform {
  required_providers {
    google = { source = "hashicorp/google", version = "~> 5.30" }
  }
}

provider "google" {
  project = var.project

    # … (full source in the desktop HTML) …

        GCP_PROJECT  = var.project
        BQ_DATASET   = "ucc_gold"
      }
    }
  }
}
```

🎯 **What Just Happened?**

You wired the full GCP production stack: state SOS drops land in `gs://*-ucc-bronze-inbound`, an Eventarc trigger fires `gcs_trigger.py` within ~2s, the trigger publishes a Pub/Sub message ordered by `state_code`, Cloud Run autoscales the orchestrator + worker pods, the daily Composer DAG reconciles missed drops + runs Bronze→Silver→Gold + loads partitioned + clustered BigQuery tables, and Terraform provisions the whole thing in one shot. Local development still works by setting `GCP_PROJECT=local` — the GCS / Pub/Sub / BigQuery clients fall back to in-memory fakes.

Advanced RAG: Hybrid Search + Jurisdiction-Specific Re-Ranking

The Bronze and Silver layers handle structured fields. The Reporting Agent answers free-text analyst questions like *"Has Acme defaulted on UCC-3 amendments in DE in the last 3 years?"* — that needs RAG over UCC regulatory docs and state filing guides. Hybrid retrieval combines BM25 keyword matching (catches statute citations like `9-515(d)`) with semantic similarity (catches conceptually related text). A jurisdiction-specific re-ranker boosts hits whose `state_code` matches the query's jurisdiction so Delaware questions get Delaware filing-guide answers first.

HYBRID_SEARCH.PY · RE_RANKER.PY

HYBRID_SEARCH.PY

```
IMPORT math
FROM collections import Counter

# Mock corpus: UCC regulatory docs + state filing guides
CORPUS = [
    {"doc_id": "UCC-9-515",
     "state_code": None,
     "title": "UCC 9-515 — Continuation",
     "text": ("A financing statement is effective for a period "
              "of five years after the date of filing. UCC 9-515 "
              "permits a continuation statement (UCC-3) within "
              "the six months immediately preceding lapse.")},
    {"doc_id": "UCC-9-310",
     "state_code": None,

    # … (full source in the desktop HTML) …

                        "bm25_score": round(bm_norm, 4),
                        "cosine_score": round(cs, 4),
                        "hybrid_score": round(hybrid, 4),
                        "query_state": state_code})
    results.sort(key=lambda x, reverse=True)
    RETURN results[:top_k]
```

RE_RANKER.PY

```
IMPORT json
IMPORT anthropic

_client = anthropic.Anthropic()
RERANK_MODEL = "claude-haiku-4-5-20251001"
JURISDICTION_BOOST = 0.25

FUNCTION _prompt(query, state_code,
            candidates):
    blocks = []
    FOR i, c IN enumerate(candidates):
        sc = c.get("state_code") or "general"
        blocks.append(
            f"[{i}] (state={sc}) {c['title']}\n"

    # … (full source in the desktop HTML) …

            c["rerank_score"] = round(base, 4)
            c["compressed_snippet"] = r.get("top_sentence",
                                            c.get("text", ""))
            enriched.append(c)
    enriched.sort(key=lambda x, reverse=True)
    RETURN enriched[:top_k]
```

🎯 What Just Happened?

You implemented the third RAG pillar. Hybrid search returns 10 candidates blending BM25 (catches "9-515" or "UCC-1") with cosine semantic similarity (catches paraphrases). The Haiku re-ranker re-orders them and contextually compresses the top sentence per result. Critically, it applies a +0.25 jurisdiction boost so a Delaware question gets the Delaware filing guide before a generic Article 9 reference. Total added cost per query: ~$0.003 — less than 0.5% of the Opus calls in the orchestrator.

Evaluation Suite: 100 Cases

TEST CASE BREAKDOWN

- **Clean filings (30):** Standard filings with no ambiguity. All entity resolutions high confidence. Validates baseline accuracy.
- **High-confidence entity resolution (15):** Name variants that should match with >85% confidence. Tests fuzzy matching and registry verification.
- **Low-confidence entity resolution (15):** Ambiguous matches that should trigger steward review. Tests HITL routing and confidence calibration.
- **Collateral classification (10):** Free-text collateral descriptions mapped to UCC categories. Tests Opus's classification accuracy on nuanced legal language.
- **Duplicate detection (5):** Filings that appear twice in different formats. Tests deduplication logic.
- **Malformed records (10):** Missing fields, encoding errors, invalid dates. Tests error handling and circuit breaker.
- **Edge cases (10):** State format changes, vector DB unavailability, concurrent batch conflicts.
- **Adversarial (5):** Injection attempts in debtor names, deliberate entity obfuscation, 5x max batch size.

Evaluation Runner (test_pipeline.py)

The 100-case suite is generated deterministically and run through the orchestrator. Accuracy targets: clean filings >95%, high-confidence resolution >90%, low-confidence resolution >75% (these go to HITL anyway), collateral classification >85%, malformed records 100% (must fail safely), adversarial 100% (must reject or quarantine).

BUILD_TEST_CASES.PY · TEST_PIPELINE.PY

BUILD_TEST_CASES.PY

```
IMPORT json
IMPORT random
FROM pathlib import Path

random.seed(2026)
STATES = ["DE", "NY", "TX", "CA", "MA", "IL", "NV", "WA"]
SUFFIX_VARIANTS = ["LLC", "L.L.C.", "Inc", "Inc.", "Corp",
                   "Corporation", "Co", "Company"]
COLLATERAL_TYPES = ["all inventory and equipment",
                    "accounts receivable",
                    "specific equipment - 2024 Caterpillar D6T",
                    "all assets of the debtor",
                    "investment property and securities",
                    "chattel paper and instruments"]

    # … (full source in the desktop HTML) …

    counts= {}
    FOR c IN s["cases"]:
        counts[c["category"]] = counts.get(c["category"], 0) + 1
    print(f"Wrote {s['total_cases']} cases)
    FOR k, v IN sorted(counts.items()):
        print(f"  {k:<30} {v}")
```

TEST_PIPELINE.PY

```
IMPORT json
IMPORT os
FROM pathlib import Path

IMPORT pytest

FROM pipeline.orchestrator import Orchestrator

SUITE_PATH = Path("evaluation/test_cases.json")

CATEGORY_FLOORS = {
    "clean_filing":             0.95,
    "entity_resolution_high":   0.90,
    "entity_resolution_low":    0.75,

    # … (full source in the desktop HTML) …

    FOR case IN suite["cases"]:
        try= orch.run(case["input"])
            IF _score_case(actual, case["expected"]) ⩾ 0.9:
                passed += 1
        except Exception= passed / len(suite["cases"])
    assert overall ⩾ 0.85, f"overall {overall:.0%} < 85%"
```

Run the harness: `python evaluation/build_test_cases.py && pytest tests/test_pipeline.py -v`

Expected output (truncated):

```
tests/test_pipeline.py::test_suite_loads PASSED
tests/test_pipeline.py::test_category_floor[clean_filing] PASSED
tests/test_pipeline.py::test_category_floor[entity_resolution_high] PASSED
tests/test_pipeline.py::test_category_floor[entity_resolution_low] PASSED
tests/test_pipeline.py::test_category_floor[collateral_classification] PASSED
tests/test_pipeline.py::test_category_floor[duplicate] PASSED
tests/test_pipeline.py::test_category_floor[malformed] PASSED
tests/test_pipeline.py::test_category_floor[edge_case] PASSED
tests/test_pipeline.py::test_category_floor[adversarial] PASSED
tests/test_pipeline.py::test_overall_accuracy PASSED
=============== 10 passed in 12.3s ===============
```

🎯 **What Just Happened?**

You closed the loop on production quality. `build_test_cases.py` generates 100 deterministic cases across 8 categories. `test_pipeline.py` runs each one through the orchestrator and asserts category-specific floors (clean filings >95%, adversarial 100%, malformed 100%). The suite is the gate that prevents regressions: any merge that drops a category below floor fails CI before it touches production.

Testing Guide

TYPE	SCENARIO	EXPECTED BEHAVIOR
`Happy`	100 clean filings batch	< 30s, all resolved, Haiku handles 70%, cost < $1.00
`Happy`	Episodic memory boosts entity confidence	Similar past case found, confidence raised above 80%, no steward needed
`Happy`	Procedural rule fires on matching registered agent	Entity resolved with zero LLM cost, deterministic rule match
`Happy`	Steward correction stored as new episode	Episode persisted, future similar cases will benefit
`Happy`	Full pipeline traced end-to-end	Dashboard shows all 8 panels with real-time metrics
`Edge`	15% malformed records in batch	Circuit breaker trips at 10%, quarantines batch, processes clean records separately
`Edge`	ChromaDB unavailable	Episodic memory degrades to keyword matching, procedural rules still fire
`Edge`	State format changes mid-batch	Ingestion agent detects schema mismatch, flags for engineering
`Adversarial`	Shell company entity obfuscation	System detects unusual patterns, flags for investigation
`Adversarial`	50,000 filing batch (5x max)	System partitions into sub-batches, processes sequentially

Compliance & Regulatory Notes

PRODUCTION COMPLIANCE REQUIREMENTS

PII handling: Filing data may contain individual names and addresses. The Reporting Agent must redact PII before generating public-facing reports. Episodic memory must not store PII — store entity IDs and resolution patterns, not raw personal data.

FCRA: If the platform's output feeds credit decisions, accuracy requirements under the Fair Credit Reporting Act apply. The 100-case evaluation suite and quality scorecards are compliance artifacts — keep them auditable.

Model audit trail: Every model routing decision, entity resolution, and steward override must be logged with timestamps. This enables regulatory audits and dispute resolution. The tracing infrastructure serves double duty: debugging AND compliance.

Data retention: Episodic memory retains resolution decisions for 730 days (2 years). Align with your organization's data retention policy and applicable state regulations.

Verify Everything Works

Run the complete production pipeline end-to-end with a single command:

COMMAND

```
python pipeline/orchestrator.py
```

Expected final output:

OUTPUT

```
============================================================
PRODUCTION UCC PIPELINE — RUN COMPLETE
============================================================
Status: complete
Total filings: 15
Elapsed: 42.31ms

  [BRONZE] {"status": "complete", "valid": 10, "quarantined": 5}
  [SILVER] {"records": 10}
  [GOLD] {"entities": 8, "cost": 0.0036}

Tracing: {"total": 3, "ok": 3, "errors": 0, "avg_duration_ms": 14.1}
```

PERFORMANCE METRICS

- **Bronze layer:** 15 filings ingested, 10 valid (67%), 5 quarantined (33%)
- **Silver layer:** 10 records normalized with standardized dates, names, and filing types
- **Gold layer:** 8 unique entities resolved, risk profiles generated
- **Tracing:** 3 pipeline stages traced, 0 errors, average 14ms per stage
- **Cost:** $0.0036 total for 15 filings = $0.00024 per filing (well under $0.02 target)
- **Circuit breaker:** Did not trip (error rate below 10% threshold for batch size)

Congratulations!

You have built a production-grade UCC data intelligence platform with all six production pillars: model routing, multi-layer memory, data quality gates, circuit breakers, distributed tracing, and a Medallion Architecture pipeline. The pipeline processes raw filings through Bronze (validation) → Silver (normalization) → Gold (entity resolution + risk profiles) with full observability at every stage.

To take this further, connect real Anthropic API calls (replace the mock resolution in `gold_layer.py`), deploy with Docker using the Dockerfile in the Deployment section, and build the 100-case evaluation harness to prove production readiness.

Troubleshooting Guide

COMMON ERRORS & FIXES

`ModuleNotFoundError: No module named 'anthropic'`
Your virtual environment is not activated. Run `source venv/bin/activate` (macOS/Linux) or `venv\Scripts\activate` (Windows), then `pip install anthropic`.

`ModuleNotFoundError: No module named 'pipeline'`
Python cannot find the package. Create `__init__.py` files in every subdirectory:
`touch pipeline/__init__.py memory/__init__.py observability/__init__.py guardrails/__init__.py routing/__init__.py`

`AuthenticationError: Invalid API key`
Your `ANTHROPIC_API_KEY` environment variable is not set or is incorrect. Run `echo $ANTHROPIC_API_KEY` (Unix) or `echo %ANTHROPIC_API_KEY%` (Windows) to verify. The key should start with `sk-ant-`.

`ImportError: cannot import name 'CircuitBreaker'`
The file `guardrails/circuit_breaker.py` is missing or has a syntax error. Ensure you completed Step 4 and saved the file.

Episodic memory returns 0 results for known entities
The mock embedding uses character frequency vectors which have lower resolution than real embeddings. Try lowering `EPISODIC_SIMILARITY_THRESHOLD` in `config.py` from 0.80 to 0.70.

Circuit breaker trips unexpectedly on small batches
The default `CIRCUIT_BREAKER_WINDOW_SIZE` is 100. If your batch is smaller than the window, the breaker only checks after all records are processed. For testing with small batches, set `window_size=5`.

Gold layer creates duplicate entities for the same debtor
This means episodic memory is not matching the name variants. Check that you are seeding episodic memory before running the gold layer, or lower the similarity threshold.

Docker build fails with "pip install" errors
Create a `requirements.txt` file with: `anthropic chromadb pydantic fastapi uvicorn celery redis httpx pytest` (one per line). Ensure it is in the project root directory.

Agent SDK Port [OPTIONAL STRETCH]

The pipeline you built uses a manual tool-use loop — `messages.create`, `stop_reason == "tool_use"`, accumulating `tool_result` blocks, and short-circuiting on `end_turn`. The **Claude Agent SDK** abstracts that loop. This stretch goal ports the Bronze→Silver transformation agent (the LLM-driven format normalizer, not the deterministic schema validator) so you can compare LOC and behavior against the manual implementation in your own code.

WHY PORT TO THE SDK?
The SDK trades fine-grained control for less code. Use it when your tools fit the MCP shape, async execution is acceptable, and the circuit breaker / retry logic can wrap the whole `query()` call rather than every iteration. Keep the manual loop when you need synchronous flow, mid-loop guardrails (e.g. PII redaction checked between tool calls), or per-tool retry semantics on flaky GCS / BigQuery operations.

Install

```
pip install "claude-agent-sdk≥0.1.0" anyio
```

Port the Bronze→Silver Transformation Agent

Create `sdk_port.py`. The same transformation tools (`detect_state_format`, `parse_filing_record`, `resolve_entity`, `flag_quality_issue`, `write_silver_record`) are wrapped with `@tool` and bundled into an in-process MCP server.

PYTHON

```
IMPORT anyio
IMPORT json
FROM claude_agent_sdk import (
    query,
    ClaudeAgentOptions,
    AssistantMessage,
    tool,
    create_sdk_mcp_server,
)
FROM agents.transform_tools import (
    detect_state_format,
    parse_filing_record,
    resolve_entity,
    flag_quality_issue,

    # … (full source in the desktop HTML) …

        "DE",
        ModelRouter(),
        Tracer(),
        CircuitBreaker(threshold=3),
    )
    print(out)
```

LOC & Behavior Comparison

ASPECT	MANUAL TOOL-USE LOOP	AGENT SDK (THIS STRETCH)
Lines per agent	~80	~30 (excluding tool wrappers)
Tool dispatch	You write the `if block.name == ...` dispatcher	SDK routes by MCP tool name
`stop_reason` / tool-use loop	You implement	SDK handles
Execution model	Sync (or your choice)	Async only
Circuit breaker hook	Inline, per iteration	Wrapper, per `query()` call
Tool result format	Plain dict you marshal	MCP `{"content": [...]}`
Mid-loop retry on tool failure	Easy — you control the loop	Harder — wrap the whole query
Visibility into raw messages	Full `messages` array	`AssistantMessage` event stream
Streaming partial responses	Set `stream=True` on `messages.create`	Native (the `async for`)
BigQuery / GCS retry on transient failure	Easy — retry the single tool result	Wrap each tool with retry decorator

DECISION HEURISTIC

Use the SDK when tools fit the MCP shape, the circuit breaker can guard the whole query, and async execution is acceptable. **Keep the manual loop when** you need synchronous control flow, per-tool-call retries (important here because BigQuery and GCS calls are flakier than Claude API calls), mid-loop PII redaction guardrails, or full audit-log visibility into the raw messages array. The Bronze→Silver pipeline keeps the manual loop because each filing is processed in a Cloud Run worker that already has Python-thread parallelism and synchronous tracing — introducing async would force a rewrite of the worker harness, and the LOC savings (~50 lines per agent) don't justify it for a five-tool agent.